



Android Application Analysis

Part 1: Static Analysis — Reading an Application Before Installation

K. Sumanth Kumar Reddy
Project Engineer, SMS Department

INTRODUCTION

About This Session

Static analysis: examining an Android application without installing or running it.

Objective

Read an APK the way an analyst does — structure, manifest and decompiled code — and recognise common vulnerability classes.

Audience

Developers, testers and security beginners who want a structured method for reviewing Android applications.

Prerequisites

Basic Android and Java familiarity. A Linux machine with apktool and jadx, and the AndroGoat sample APK.

Outcome

Five findings identifiable from the file alone: hardcoded secrets, insecure storage, exported components, weak cryptography and misconfigured flags.

AGENDA

What This Session Covers

A guided walkthrough of static analysis — from APK structure to real vulnerability classes — without installing or executing the application.



Each phase builds on the last — reading the file, then reading the code, then judging what the code does wrong.

CONCEPTS

What Is Application Security Analysis?

A pre-requisite look at the discipline this session is built on.

In simple terms: a security analyst examines an app the way a reviewer examines a building before it opens — checking doors, locks and wiring before anyone else walks in.



Two Complementary Approaches

Security analysis is generally split into two disciplines that answer different questions about the same application.



IN FOCUS

Static Analysis (SAST) — First

Examines the APK file and decompiled source without ever running it. Finds hardcoded secrets, insecure logic and misconfiguration by reading.



NEXT UP

Dynamic Analysis (DAST) — Next

Observes the application while it executes on a device or emulator, using tools such as Frida to inspect real runtime behaviour.

INDUSTRY FRAMEWORK

OWASP Mobile Security Guidelines

The reference standards this session's findings are mapped against throughout — look for the tags on later slides.



STANDARD

MASVS

Mobile Application Security Verification Standard — defines the security requirements an app should meet, grouped into categories such as STORAGE, CRYPTO, NETWORK and PLATFORM.



GUIDE

MASTG

Mobile Application Security Testing Guide — the practical manual: how to actually test for each MASVS requirement, static and dynamic.



TOP 10

OWASP Mobile Top 10

The ten most common and impactful mobile risk categories — the fastest way to prioritise findings by real-world relevance.

Reference: owasp.org/www-project-mobile-app-security · Findings in this session are tagged, e.g. MASVS-STORAGE, MASVS-CRYPTO, MASVS-PLATFORM.

WHY THIS MATTERS

The Scale of the Problem

Recent cybercrime and mobile-malware figures — the real-world backdrop for this session.



28.15 L

Cybercrime cases reported to India's
National Cyber Crime Reporting Portal
in 2025



₹22,812 Cr

Cyber-fraud losses reported in India in
2024 — up 895% since 2022



14 M+

Android malware & adware attacks
blocked worldwide in 2025 (Kaspersky)



2,55,090

New Android banking-trojan install
packages detected in 2025

Sources: National Cyber Crime Reporting Portal / Ministry of Home Affairs, 2025 · Kaspersky Mobile Malware Report, 2025. Figures are the latest publicly available at the time of this session.

Every statistic above traces back to an application that shipped with a weakness this course is designed to surface first.

MOTIVATION

Why Application Analysis Is Necessary

The what, why, obstacles and outcomes of security analysis — before moving into technique.



WHAT

What It Is

A structured review of an application's code, configuration and behaviour to identify exploitable weaknesses against a known standard, such as OWASP MASVS.



WHY

Why It Is Needed

Release cycles are fast and security review is often an afterthought. A single flaw, shipped once, is replicated across every install of the app.



CHALLENGES

The Challenges

Obfuscated code, large codebases, time pressure, and a constantly evolving Android OS make manual review slow and easy to under-scope.



OUTCOME

What It Solves

Surfaces hardcoded secrets, insecure storage, weak cryptography and risky configuration — before release, not after a breach.

1 · APK ANATOMY

An APK Is Simply a ZIP Archive

An APK (Android Package Kit) is the file Android uses to install an app — the starting point for every static analysis engagement.

```
cdac@cdac-Precision-5860-Tower:~/Desktop/Training/Android_application_analysis/Assignments$ unzip Assignment-1.apk -d AssignnOut
Archive: Assignment-1.apk
  inflating: AssignnOut/AndroidManifest.xml
  inflating: AssignnOut/META-INF/CERT.RSA
  inflating: AssignnOut/META-INF/CERT.SF
  inflating: AssignnOut/META-INF/MANIFEST.MF
  inflating: AssignnOut/classes.dex
  inflating: AssignnOut/res/layout/activity_main.xml
  inflating: AssignnOut/res/menu/menu_main.xml
  extracting: AssignnOut/res/mipmap-hdpi-v4/ic_launcher.png
  extracting: AssignnOut/res/mipmap-mdpi-v4/ic_launcher.png
  extracting: AssignnOut/res/mipmap-xhdpi-v4/ic_launcher.png
  extracting: AssignnOut/res/mipmap-xxhdpi-v4/ic_launcher.png
  extracting: AssignnOut/resources.arsc
cdac@cdac-Precision-5860-Tower:~/Desktop/Training/Android_application_analysis/Assignments$ ls AssignnOut/
AndroidManifest.xml  classes.dex  META-INF  res  resources.arsc
```



classes.dex

Compiled application code



resources.arsc + res/

Text, layouts and images



META-INF/

Signing certificate

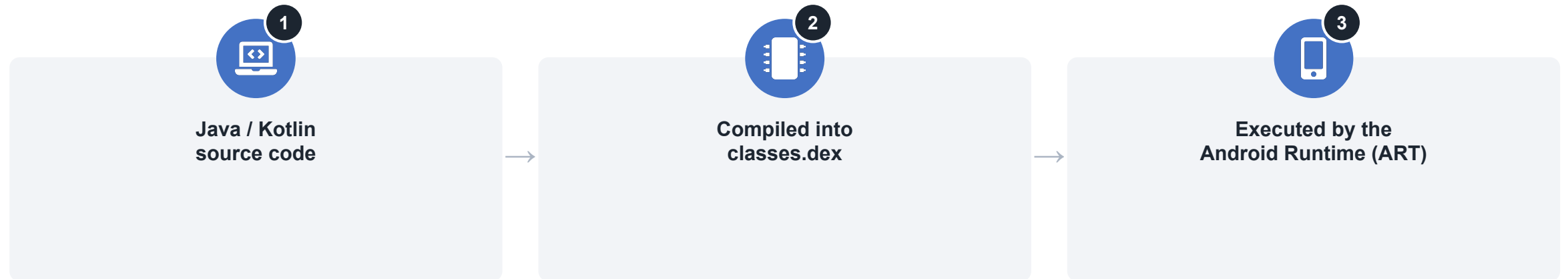
unzip -d on a real APK — every file the developer shipped, listed out.

Renaming the extension to .zip and extracting it exposes every file the developer shipped — no tooling required.

2 · DEX, RESOURCES & MANIFEST

classes.dex — The Compiled Application Logic

Every class, method and instruction the application has, compiled into one file.



What is ART?

ART (Android Runtime) is the program built into every Android phone that reads classes.dex and actually runs the app.

2 · DEX, RESOURCES & MANIFEST

resources.arsc — What the App Shows on Screen

Purpose: one file bundling everything the user sees — text, colours, layouts and images — each given a numeric ID.



strings.xml
(text)

```
<string name="app_name">AndroGoat - Insecure App (Kotlin)</string>
```



resources.arsc
(compiled)

```
id 0x7f0b0036 → “AndroGoat - Insecure App (Kotlin)”
```



AndroidManifest.xml
(usage)

```
android:label="@string/app_name" – the ID is used, not the text
```

Real values from AndroGoat — the application looks up text and images by ID at runtime, never by name.

2 · DEX, RESOURCES & MANIFEST

AndroidManifest.xml — The App's ID Card

Purpose: one file that declares every screen, permission and entry point the app has.



The first file worth reading

Before opening a single line of Java — it summarises the whole app in one place



Permissions

What the app is allowed to access on the device



Components

Activities, services, receivers and providers declared



Security Flags

debuggable, allowBackup, exported, cleartext traffic

Real example, from a decoded manifest:

```
2 <uses-permission android:name="android.permission.INTERNET"/>
3 <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
4 <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
```

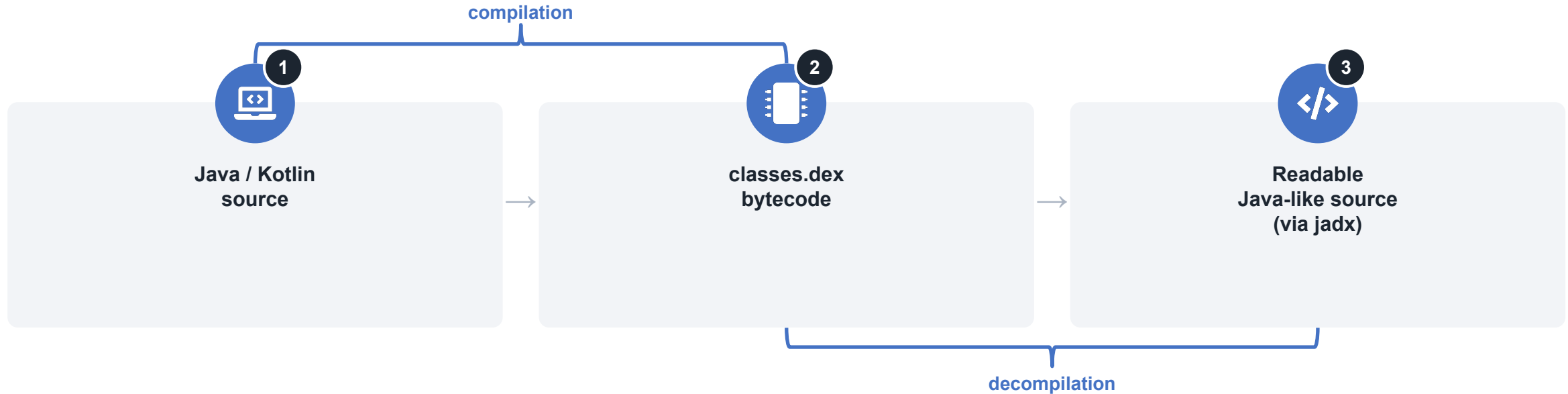
```
6 <activity android:name="owasp.sat.agoat.SplashActivity">
7   <intent-filter>
8     <action android:name="android.intent.action.MAIN"/>
9     <category android:name="android.intent.category.LAUNCHER"/>
```

```
android:debuggable="true"
android:allowBackup="true"
```

3 · DECOMPIlation & REVERSE ENGINEERING

Reconstructing Readable Source from Bytecode

Decompiling walks the build process backward — bytecode becomes Java-shaped source again.



One instruction, two views

```
smali (apktool)
invoke-virtual {p0},
    Ljava/lang/String;->trim()V
```

```
decompiled (jadx)
str.trim();
```

3 · DECOMPILE & REVERSE ENGINEERING

jadx vs. apktool — Complementary Tools

Two tools, two trade-offs — both used together in this session.



OPEN SOURCE

jadx

Decompiles classes.dex into near-Java source. Best for reading logic and locating strings. Falls back to smali when a class cannot be decompiled.

github.com/skylot/jadx



OPEN SOURCE

apktool

Unpacks resources and the real manifest XML. Never produces Java, but never fails either — the reliable second opinion.

apktool.org

Manual reading with jadx and apktool is the method for this session. At scale, teams add automated SAST scanners on top — MobSF and QARK run the same checks across hundreds of APKs and print a report.

TOOLING

Open-Source Tools Used in This Session

Every tool in this course is free and open source — no licensing barrier to replicating this workflow.

apktool

OPEN SOURCE

Unpacks resources and
decodes the real manifest
XML

apktool.org

jadx

OPEN SOURCE

Decompiles DEX to
near-Java source for code
reading

github.com/skylot/jadx

MobSF

OPEN SOURCE

Automated scanner that
produces a full static report

mobsf.github.io

Ghidra

OPEN SOURCE

Disassembler and decompiler
for native (.so) code

ghidra-sre.org

TOOL WALKTHROUGH · APKTOOL

Unpacking Resources with apktool

apktool never produces Java — it decodes resources and the manifest, and falls back to smali for code.

OPEN SOURCE

```
cdac@cdac-Precision-5860-Tower:~/Desktop/Training/Android_application_analysis/Assignments$ apktool d Assignment-2.apk -o Assign2Out
I: Using Apktool 2.12.1 on Assignment-2.apk with 8 threads
I: Baksmaling classes.dex...
I: Loading resource table...
I: Decoding file-resources...
I: Loading resource table from file: /home/cdac/.local/share/apktool/framework/1.apk
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying lib...
I: Copying unknown files...
cdac@cdac-Precision-5860-Tower:~/Desktop/Training/Android_application_analysis/Assignments$ ls Assign2Out/
AndroidManifest.xml  apktool.yml  lib  original  res  smali  unknown
```

apktool d on a real assignment APK — decoded resources, manifest, and a smali/ folder.

Reference: apktool.org



smali/

Disassembled bytecode, always present, even when jadx cannot decompile a class



res/ + Manifest

Decoded to plain, readable form

TOOL WALKTHROUGH · JADX-GUI

Reading Java-Like Source with jadx-gui

Launched against a target APK — near-Java source, easy to read.

OPEN SOURCE



TOOL WALKTHROUGH · MOBSF

Automating the Analysis with MobSF

Mobile Security Framework — automated security analysis for Android, iOS and Windows apps.

OPEN SOURCE



Decompiles and scans automatically

Hardcoded secrets, security misconfigurations and known vulnerabilities



Reviews the manifest

Flags excessive permissions and improperly exported components



Checks third-party libraries

Against the CVE database for outdated or vulnerable versions



Flags common code-level issues

Insecure storage, weak cryptography, SQL injection, insecure communication

A live scan runs at mobsf.live rather than a local install — same engine, no setup.

Reference: [mobsf.github.io](https://github.com/mobsf/mobsf)

4 · CODE OBFUSCATION

Obfuscation: Slowing Down the Analyst

Renames and reshapes code so it still runs identically, but reads like nonsense.



Names shrink to letters

LoginActivity becomes a — classes, methods and fields all shrink



Strings get encoded

Only decoded at runtime, by a helper method



Built in automatically

Applied to most release builds via R8 or ProGuard



The Limit of Obfuscation

It slows reading down — it does not remove the logic. The behaviour is still fully present to be found, given enough time.

4 · CODE OBFUSCATION

Technique 1 — Rename

Classes, methods and fields shrink to single letters — the default behaviour of R8 and ProGuard.

```
// before
boolean checkPassword(String input) {
    return input.equals(storedPassword);
}
```

```
// after obfuscation
boolean a(String b) {
    return b.equals(c);
}
```

It slows down reading; it does not remove the logic — the behaviour is still fully there to find.

4 · CODE OBFUSCATION

Technique 2 — String Encryption

Text literals are scrambled in the file, and only decoded back by a helper method while the app is running.

```
// before
String password = "admin123";

// after – unreadable in the decompiled source
byte[] enc = {0x0C, 0x2E, 0x2D, 0x29, 0x39, 0x2D, 0x39, 0x2D};
String password = decrypt(enc);
```

4 · CODE OBFUSCATION

Technique 3 — Control-Flow Flattening

Extra branches that always evaluate the same way are added, so the logic is harder to follow without changing what it does.

```
// before
if (isPremium) {
    unlockFeature();
}

// after — condition is always true, harder to see why
if ((7 * 3 == 21) == true) {
    unlockFeature();
} else {
    wasteTime();
}
```

4 · CODE OBFUSCATION

Technique 4 — Resource Obfuscation

File and resource names are scrambled too, not just the code — the filename itself stops being a clue.

```
// before  
res/layout/activity_main.xml  
res/values/strings.xml  
  
// after – filenames carry no meaning  
res/a/b.xml  
res/a/c.xml
```

4 · CODE OBFUSCATION

Technique 5 — Native Packing

The sensitive logic moves out of Java entirely, into a compiled .so library — a decompiler like jadx cannot follow it there.

```
// before
boolean checkLicense() {
    return key.equals(realKey);
}

// after — logic left Java entirely
native boolean checkLicense();
// body now lives in libcore.so, called via JNI
```

4 · CODE OBFUSCATION

Commercial Obfuscation & App-Protection Tools

Beyond free R8/ProGuard — paid platforms typically used in production BFSI and DRM-sensitive apps.



COMMERCIAL

DexGuard (Guardsquare)

Commercial superset of ProGuard — adds string and asset encryption, class encryption, and runtime self-protection (RASP).



COMMERCIAL

Allatori

Java/Android obfuscator combining control-flow obfuscation, string encryption and tamper-detection watermarking.



COMMERCIAL

Digital.ai Application Protection (Arxan)

Enterprise-grade app shielding — anti-tamper, anti-debug and code obfuscation for finance and DRM use cases.



COMMERCIAL

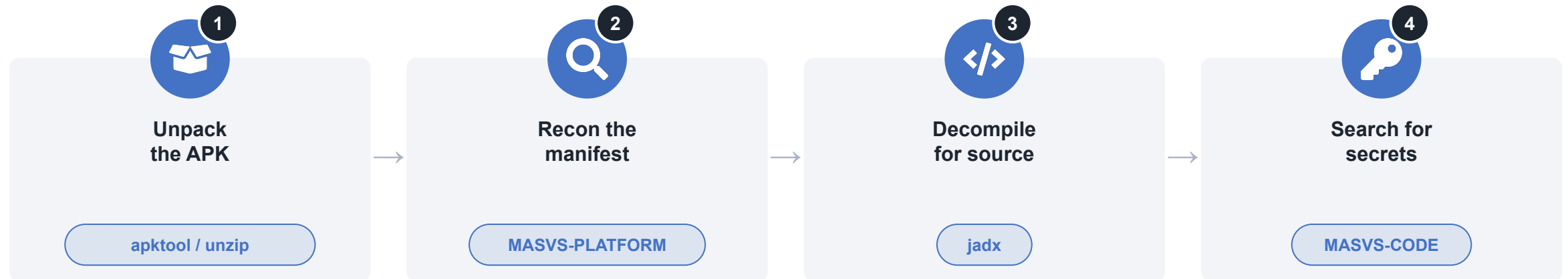
Zimperium zShield / zDefend

Runtime Application Self-Protection (RASP) layered on top of obfuscation — detects and reacts to tampering live.

5 · STATIC ANALYSIS METHODOLOGY

A Repeatable Methodology — Phase 1: Discovery

The same sequence for a five-screen practice app or a five-hundred-activity banking app.

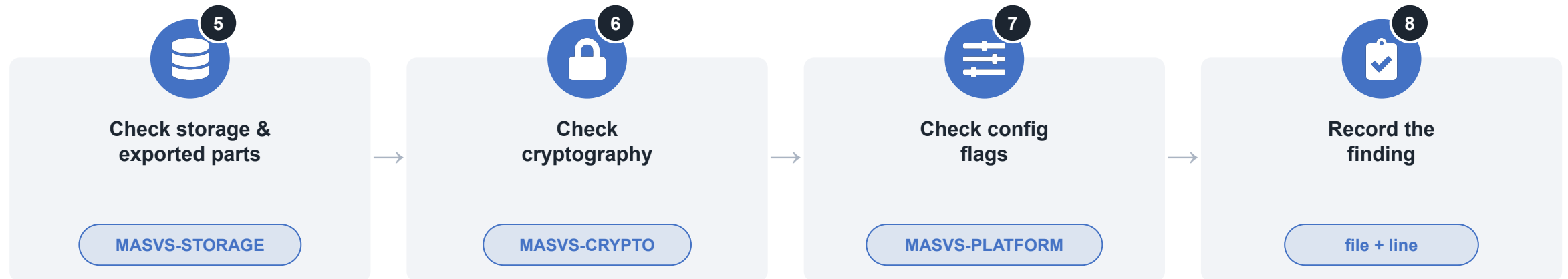


Discovery establishes what the application contains before any finding is judged.

5 · STATIC ANALYSIS METHODOLOGY

A Repeatable Methodology — Phase 2: Verification

The second half confirms whether each candidate finding actually holds up.



Every finding needs a file and a line number — verification is what turns a suspicion into a report entry.

6 · IDENTIFYING HARDCODED SECRETS

A Compiled-In Value Is Not a Secret

A secret only protects something if it is not shipped inside the file an attacker already holds.

MASVS-CODE

```
// HardCodeActivity.kt – found in the decompiled source
val promoCode = "NEW2019"

if (userInput == promoCode) {
    price.text = "0"    // full discount applied
} else {
    price.text = "2000" // incorrect code entered
}
```



Why is it there? Developers hardcode values like this for convenience while building or testing — a quick way to check the logic without a real server — and forget to remove them before release.

Hardcode Issue

Objectives:

1. Find out how/where Promocode is hardcoded.
2. Enter Promocode to get below product for free



Qty: 1 Price: 2000

Enter Promocode here
AndroGoat, running on-device — the same promo screen.

6 · IDENTIFYING HARDCODED SECRETS

Detecting Hardcoded Secrets at Scale

A single search pattern, run against the entire decompiled source tree.

```
$ grep -rniE "api_key|password|secret|token" out/sources/
```



Dead code counts too

An unused field still ships inside the APK — a secret does not need to be reachable at runtime to count as exposed.

7 · INSECURE STORAGE & EXPORTED COMPONENTS

Insecure Storage — Plain-Text Preferences

Saving sensitive data as plain text instead of encrypting it.

MASVS-STORAGE

```
// InsecureStorageSharedPrefs.kt
getSharedPreferences("users", 0).edit()
    .putString("username", user)
    .putString("password", pass)
    .commit();
```



Where it lands

shared_prefs/users.xml — plain text, readable straight from a backup if allowBackup="true".

Shared Preferences -Part1

Objectives:

1. What is Shared Preferences?
2. Identify how/where sensitive information is stored
3. Vulnerable Code
4. Risk associated with it

Username

Password

The real login form in AndroGoat.

Why is it there?

Local storage looks private to the developer, so encrypting it feels unnecessary — until a backup or a rooted device proves otherwise.

7 · INSECURE STORAGE & EXPORTED COMPONENTS

Insecure Storage — Local Database

SharedPreferences is not the only place data sits in plain text — a local SQLite database is just as easy to pull and open.

MASVS-STORAGE

```
# works whenever debuggable="true" makes run-as available
$ adb shell run-as owasp.sat.agoat ls databases/
aGoat

$ adb exec-out run-as owasp.sat.agoat \
    cat databases/aGoat > aGoat.db

$ sqlite3 aGoat.db
sqlite> .tables
users
sqlite> SELECT * FROM users;
```

SQLite

Objectives:

1. What is SQLite Database?
2. Identify how/where sensitive information is stored
3. Vulnerable Code
4. Risk associated with it

Username

Password

This module's real entry screen.

Same reason as SharedPreferences: a local database still feels private to the developer — the device just makes it easy to prove otherwise.

7 · INSECURE STORAGE & EXPORTED COMPONENTS

Exported Components — Reachable Without Code

android:exported="true" with no permission means any app — or adb — can talk to it directly.

MASVS-PLATFORM

Generic syntax

```
$ adb shell am start -a <ACTION> -d "<deep_link_uri>"
```

Applied to AndroGoat's exported activity

```
$ adb shell am start -a android.intent.action.VIEW -d "androgoat://vulnapp"
```

Why is it there? Components are often exported on purpose, to let other apps use them — but the developer forgets to add a permission check, or to set exported="false" on the ones that should stay private.

7 · INSECURE STORAGE & EXPORTED COMPONENTS

Exported Components — Services & Receivers

The same idea reaches background components too — no screen, no source code, still exploitable.

MASVS-PLATFORM

SERVICE

```
# generic
$ adb shell am startservice \
    -n <package>/<.ServiceName>

# AndroGoat
$ adb shell am startservice -n \
    owasp.sat.agoat/.DownloadInvoiceService
```

RECEIVER

```
# generic
$ adb shell am broadcast \
    -n <package>/<.ReceiverName>

# AndroGoat
$ adb shell am broadcast -n \
    owasp.sat.agoat/.ShowDataReceiver
```

8 · WEAK CRYPTOGRAPHY

A Hardcoded SecretKeySpec

Encryption is only as good as the key behind it — here the key is sitting in plain sight.

MASVS-CRYPTO

```
// weak – identical in every install
private static final String KEY =
    "1234567890123456";
SecretKeySpec keySpec =
    new SecretKeySpec(KEY.getBytes(), "AES");
```

```
// secure – generated per device
KeyGenerator kg = KeyGenerator
    .getInstance("AES", "AndroidKeyStore");
kg.init(new KeyGenParameterSpec.Builder(
    "appKey", PURPOSE_ENCRYPT|PURPOSE_DECRYPT)
    .setBlockModes(BLOCK_MODE_GCM).build());
```

Why is it there? Writing the key as a constant is the simplest way to get encryption working during development — and it is never migrated to secure storage afterwards. The fix is not a stronger algorithm; it is never writing the key down in the first place.

8 · WEAK CRYPTOGRAPHY

The Initialisation Vector Never Changes Either

CBC mode needs a fresh, random IV every time — reuse one and patterns start leaking through again.

MASVS-CRYPTO

```
// weak – same 16 bytes, every call
private static final byte[] IV =
    new byte[16]; // all zero bytes
IvParameterSpec ivSpec =
    new IvParameterSpec(IV);
```

```
// secure – random IV, generated fresh
byte[] iv = new byte[12];
new SecureRandom().nextBytes(iv);
GCMParameterSpec spec =
    new GCMParameterSpec(128, iv);
cipher.init(ENCRYPT_MODE, key, spec);
```

Why is it there? A static IV is often copied from an online example without understanding why it must change every time. Same key + same IV + same plaintext = same ciphertext — CBC starts leaking the way ECB does.

9 · MISCONFIGURATIONS

Flags That Expose the Device

None of these need a bug to exploit — just the manifest set the wrong way. Usually left on by accident after development.



MASVS-RESILIENCE

debuggable="true"

Lets a debugger attach, and unlocks run-as — an adb command that reads any app's private files without root, as easily as reading its own.



MASVS-STORAGE

allowBackup="true"

adb backup can copy the app's private data off the device to a laptop — no root required.

9 · MISCONFIGURATIONS

Flags That Expose the Network & Components

The rest of the checklist — same idea, different surface.



MASVS-NETWORK

usesCleartextTraffic="true"

Plain HTTP is allowed — traffic is trivially sniffable or MITM-able.



MASVS-PLATFORM

exported="true"

Component is launchable by any app, or directly via adb, if no permission guards it.



MASVS-PLATFORM

targetSdkVersion (low)

Misses every OS security default added since that API level.

9 · MISCONFIGURATIONS

Turning Two Flags Into a Data Pull

debuggable="true" and allowBackup="true" — combined, they bypass root entirely.

```
# run-as is normally blocked per-app –  
# debuggable=true switches it on, no root  
$ adb shell run-as owasp.sat.agoat \  
    cat shared_prefs/users.xml  
  
# or attach a real Java debugger:  
$ adb shell am set-debug-app -w \  
    owasp.sat.agoat
```

```
# pulls private data off the device –  
# no root, no exploit needed  
$ adb backup -f agoat.ab owasp.sat.agoat  
  
$ dd if=agoat.ab bs=1 skip=24 | \  
    zlib-flate -uncompress > agoat.tar  
$ tar xf agoat.tar
```

10 · WEBVIEW SECURITY

WebView — A Browser Inside the App

A WebView is a browser inside the app, so web flaws become app flaws. In jadx, search for “WebView”.



MASVS-NETWORK

loadUrl("http://...")

Content loaded over plain HTTP, or from a value the user controls, can be tampered with in transit.



MASVS-PLATFORM

setJavaScriptEnabled(true)

Loaded pages can run JavaScript. Risky when the page comes from an untrusted or attacker-influenced source.



MASVS-PLATFORM

addJavascriptInterface()

Exposes a Java object to page JavaScript. A malicious page may call its methods and reach app functionality.

NATIVE LAYER

One Layer Below Java: Native Libraries

A simple, top-level look at native code inside an APK.



lib/<abi>/*.so files

Compiled C/C++ code, loaded by the app with `System.loadLibrary()`



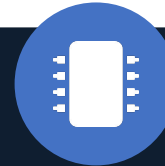
jadx cannot reach it

It only decompiles `classes.dex` — reading native code needs a different tool, such as Ghidra



Called from Java via JNI

A bridge that lets Java code call into the native library and back



Not Covered Here

Reading native code needs its own tools and its own session. This slide is only meant to show that this layer exists.

WORKED EXAMPLE

One Tiny Application, Five Findings

Everything from this session, mapped onto a single real app: AndroGoat.

AndroGoat - Insecure App (Kotlin)

AndroGoat is purposely developed open source vulnerable/insecure app using Kotlin. Security Testers/Professionals/Enthusiasts, Developers...etc. can use this application to understand and defend the vulnerabilities in Android platform. This is the first vulnerable app developed using Kotlin.
If you are looking to learn Android Application Security Testing then AndroGoat is a perfect solution.
Happy Learning

NETWORK INTERCEPTING

UNPROTECTED ANDROID COMPONENTS

INSECURE DATA STORAGE

INPUT VALIDATIONS

SIDE CHANNEL DATA LEAKAGE

HARDCODE ISSUE

AndroGoat's real home screen.



MASVS-RESILIENCE

debuggable="true"

Should never ship this way.



MASVS-STORAGE

allowBackup="true"

Private data is one adb command away.



MASVS-PLATFORM

Exported service

DownloadInvoiceService, no permission required.



MASVS-CODE

Hardcoded promo code "NEW2019"

Directly in source.



MASVS-STORAGE

Plain-text password in prefs

No encryption at all.

WRAP-UP

Static Analysis, In Summary

Everything in this session came from reading the APK, manifest and decompiled code — nothing installed, nothing executed.



Obfuscation slows reading down

It does not hide behaviour — every technique remains reversible with enough time



Five findings now identifiable

Hardcoded secrets, insecure storage, exported components, weak cryptography, misconfigured flags



Next: what static analysis cannot show

Runtime behaviour, and dynamic tools such as Frida



Questions?

Thank You